



A Project Report on
Democratizing Medical Diagnosis: A Visual Question Answering (VQA) based Approach

Submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF ENGINEERING
IN
INFORMATION SCIENCE AND ENGINEERING

By

Yash Jaiswal	1NT18IS185
Sowmya Sri P	1NT18IS158
Shubham Saini	1NT18IS150
Somya Srivastava	1NT18IS155

Under the Guidance of

Dr. Archana Mathur

Associate Professor

Department of Information Science and Engineering

Nitte Meenakshi Institute of Technology, Bengaluru - 560064



DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING
(Accredited by NBA Tier-1)

2021-2022

NITTE MEENAKSHI INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION, AFFILIATED TO VTU, BELGAUM)

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

Accredited by NBA Tier-1



CERTIFICATE

Certified that the project work entitled "**Democratizing Medical Diagnosis: A Visual Question Answering (VQA) based Approach**" carried out by Ms. **Somya Srivastava**, USN **1NT18IS155**, a bonafide student of **Information Science and Engineering** department and **Nitte Meenakshi Institute of Technology** in partial fulfillment for the award of Bachelor of Engineering in **Information Science and Engineering** of the Visvesraya Technological University, Belgaum during the year **2021 - 22**. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the Report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Dr. Archana Mathur (Project Guide) Dr. Mohan S. G. (HOD, ISE Dept.) Signature of the Principal

External Viva

Name of the examiners

Signature with date

1.

2.

NITTE MEENAKSHI INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION, AFFILIATED TO VTU, BELGAUM)

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

Accredited by NBA Tier-1



DECLARATION

I, Somya Srivastava (1NT18IS155), bonafide student of Nitte Meenakshi Institute of Technology, hereby declare that the project entitled "Democratizing Medical Diagnosis: A Visual Question Answering (VQA) based Approach" submitted in partial fulfillment for the award of Bachelor of Engineering in Information Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2021-2022 is my original work and the project has not formed the basis for the award of any other degree, fellowship or any other similar titles.

Signature of the Student with Date

Place: Bangalore

Date:

ABSTRACT

Decision Support Systems play an integral role in today's diagnosis procedures adopted by doctors. But, often, doctors require intelligent systems that can answer image-oriented queries of the clinician. Visual Question Answering (VQA) systems in the Medical Domain (MedVQA) take into context both - the medical imagery and the image-oriented questions - to answer the queries of the clinicians. This report discusses 2 architectures for implementing VQA systems and aims to serve a thorough comparative study for the two architectures. The first approach uses the Multi-modal Factorized Bilinear (MFB) Pooling architecture and the second approach uses a Hierarchical Image-Question Co-attention model. Both approaches fall under the umbrella of attention-based VQA models. The ImageCLEF 2019 dataset is employed for training and testing the models. The outcome obtained for each approach are shown and discussed. The results for the second approach are two folds: the first attempt shows the loss values obtained at each epoch without using any specific training strategy; the second attempt shows the loss values obtained at each epoch wherein learning rate decay strategy is employed. The first approach, whilst adept at merging two feature maps together, had a complex architecture that took longer runtimes for training and had ambiguous results whereas the second approach which boasted a much simpler architecture and relatively faster training time showed an accuracy score of 42% in just 40 epochs. There is a definite scope for improvement in the proposed approaches by utilizing better strategies for hyperparameter tuning, proper training and experimenting with different feature extraction modules for images. These improvement strategies are also discussed briefly.

ACKNOWLEDGMENT

Firstly, I would like to sincerely thank my mentor, Dr. Archana Mathur, for her insightful remarks, wise counsel, and constant encouragement, all of which have been of great assistance to me throughout my research. Her vast knowledge, extensive experience, and professional skills in the field of deep learning has made it possible for me to effectively complete this research. The success of this endeavor would not have been possible without her assistance and direction.

I am also grateful to respected Dr. Mohan S.G., Head of Department, Information Science and Engineering, for permitting me to use all the necessary facilities. I am also thankful to all the other faculties and staff members of my department for their kind cooperation and help. Finally, I would like to thank the management and staff at Nitte Meenakshi Institute Of Technology for their continuous support.

Contents

1	Introduction	1
2	Literature Review	3
3	Exploratory Data Analysis (ImageCLEF 2019)	5
4	Requirement Specification	7
4.1	Project Perspective	7
4.2	Software Requirement	7
4.3	Hardware Requirement	7
4.4	Functional Requirements	7
4.5	Non-Functional Requirements	7
4.5.1	Usability Requirement	7
4.5.2	Availability Requirement	8
4.5.3	Efficiency Requirement	8
4.5.4	Accuracy	8
4.5.5	Performance Requirement	8
4.5.6	Reliability Requirement	8
5	Framework and System Design	9
5.1	Image Feature Extraction Module	10
5.2	Question Semantic Encoder	10
5.3	Feature Fusion	10
5.3.1	Multi-modal Factorized Bilinear Pooling model with Co-attention	11
5.3.2	Hierarchical Question-Image Co-attention model	11
6	Implementation	15
6.1	Approach 1: Multi-modal Factorized Bilinear Pooling model with Co-attention	16
6.2	Approach 2: Hierarchical Question-Image Co-attention model	19
7	Testing	23
7.1	Learning Rate Decay Technique	24
8	Results	25
8.1	Approach 1: Multi-modal Factorized Bilinear Pooling with Co-attention	25
8.1.1	Outcome for module 1	25
8.1.2	Outcome for module 2	25
8.1.3	Outcome for module 3	26
8.2	Approach 2: Hierarchical Question-Image Co-attention mechanism	27
9	Conclusion and Future Scope	29

List of Figures

3.1	Image Repository	5
3.2	Question Answer Pairs	5
3.3	QA pairs with respect to a single image	6
3.4	Category-wise QA distribution	6
5.1	MFB architecture	11
5.2	Hierarchical Question-Image Co-attention model framework	12
6.1	MFB with Co-Attention model architecture using VGG16 and BERT	16
6.2	VGG16 model (with custom output layers)	17
6.3	Block diagram for VGG16 model with block-1 convolutional-2 output layer	17
6.4	Code snippet for custom VGG16 model	17
6.5	Output for individual VGG16 block layers	18
6.6	Importing base BERT model	18
6.7	Processing BERT end result	18
6.8	Semantic encodings obtained from BERT	19
6.9	Code implementation for MFB	19
6.10	Code implementation for calculating affinity matrix	20
6.11	Code implementation for computing attention feature maps	20
6.12	Code implementation for computing the final output using the hierarchical representation	21
6.13	Output vectors produced by model	21
7.1	Loss-epoch graph obtained during model training	23
7.2	Log file for all epochs trained during model training runtime	24
7.3	Loss-epoch graph obtained during model training using LR-decay strategy	24
8.1	VGG16 module outcome	25
8.2	BERT module outcome	26
8.3	MFB module outcome	26
8.4	Model outcome for approach 2	27
8.5	Loss and accuracy scores for approach 2	27

Chapter 1

Introduction

Medical scans are tough to assimilate for people without proficiency. The lack of medical practitioners around the world causes exhaustion due to the zillions of cases, inducing human mistakes during the prognosis. In such conditions, having a ‘second opinion’ can be advantageous in uplifting the trust of the decision-maker. This gap can be bridged using Diagnostic Decision Support Systems (DDSS) [1][2] but current DDSS are limited in their use-cases since they cannot answer complex, image-oriented questions of the clinician.[3]

Computers are expected to be able to spontaneously understand the pragmatics of images and natural languages in the future, thanks to recent advances in the fields of computer vision and NLP (natural language processing). These advancements have also sparked new research topics such as image-text retrieval[4], image captioning[5], and visual question answering[6]. In contrast to image-text retrieval and image captioning (which only require basic algorithms to search for or generate a free-form text illustration for a specific image), visual question answering (VQA) is a more difficult task that necessitates a thorough understanding of both the images and the natural language questions, as well as complex reasoning to correctly predict the best-fit answer. In some cases, the VQA task can function as a globalisation of image captioning and image-text retrieval. Thus, developing fruitful VQA algorithms that can achieve near-human performance is a critical step toward empowering artificial intelligence in general.

Visual question answering (VQA) is a collaborative research issue that demands sufficient base on computer vision and natural language processing approaches. The question has notarized substantial advancement towards complex visual interpretation and semantic interpretation tasks, owing to the accessibility of large-scale standard data-sets [7] and swift development of deep learning procedures. It lets machines think rationally covering and beyond language and vision like humans and is perceived as one vital assessment for machine intelligence. It has come out as an eminent interdisciplinary experimentation in both varsities and commerce. To give accurate responses to discernible questions about an image, the machine requires an interpretation of both the image and the question. Lately, visual attention based models have been examined for VQA, where the attention structure often generates a spatial map accentuated image regions that are appropriate to form a response to the question. [8]

In the domain of healthcare, medical practitioners often use radiology scans of their patients for diagnostic purposes. They, therefore, require intelligent DDSS which can answer questions specific to a medical scan (like CT-scans, X-Rays, etc.)[9][10]. The foregoing diagnostic decision support systems (DDSS) either consider a medical scan or a medical question so as to execute a particular piece of work, like kidney stone detection (via images of medical scans) or a simple mental health chatbot (using natural language processing). Despite the fact that such systems are productive, medical practitioners often require answers to questions pertaining to a medical scan or image, such as “what is the organ shown in this x-ray scan?”, “what abnormality is seen in the image?” , “in what plane is this image?”, etc. For such scenarios, there is a demand for a mechanism that considers both - the image along with the natural-language questions of the medical practitioners.

The proposed method is to create a Diagnostic Decision Support System (DDSS) . The input given to this system would be a medical image and a corresponding question associated with that image. [11]This subsequently generates a natural language response as the output. DDSS makes use of a Visual Question Answering (VQA) mechanism which functions at the convergence of computer vision and natural-language-processing

methods. A mechanism of this worth would give a helping hand to the medical practitioners and be of service to the common population in solving their complex questions. With this method, the objective of the project is to initiate and move ahead in democratizing medical diagnosis and imparting identical healthcare opportunities for all. VQA is used as a confirmatory step in the diagnosis. It is an algorithm that takes 2 inputs, one an image, and the other a natural language question based on it, and then it generates a natural language response as the output.

The solution that is put forward consists of mainly three modules, which are : image feature extraction, question semantic encoder, and feature fusion using a hierarchical co-attention mechanism. The image feature extraction module takes as input an image, extracts important and relevant features from it and provide them as an output. The first part of our solution includes using resnet18 to extract the image features. The question semantic encoder takes the natural language question as an input and extracts important features and feeds them as an input to the next module. LSTM[12] is used to extract the embeddings from the question. The features of the image and the question obtained from the previous two modules are given as input to the feature fusion module. After we receive the co-attended image and the question features by using the co-attention mechanism, it is sent into a convolutional layer followed by a softmax layer which then predicts the answer. The features from the previous two modules are fused together to predict answers for VQA model. The VQA model is then trained and fine-tuned to obtain enhanced accuracy.

Chapter 2

Literature Review

This paper [13] talks about how VQA systems should use a Question Segregation (QS) method to segregate questions into 2 types: (i) Binary (Yes/No) response-type questions; (ii) Descriptive response-type questions

The author's in this paper [14] have explored the use of the unsupervised Denoising Auto-Encoder (DAE) and the supervised Meta-Learning. The advantage of DAE is to leverage the large number of unlabeled images while the advantage of Meta-Learning is to learn meta-weights that quickly adapt to VQA problems with limited labeled data.

Bilinear Combination-based models have been shown to outperform traditional VQA line models, but are of higher magnitude. These models have shown to outperform traditional vqa models .The paper [15] takes into consideration both the image and the question features and use co-attended image and question features. This helps the model better understand the image and the question features.

This paper [16] discusses the challenge of assessing the similarity of picture collections with variable-quantity, quality, and un-ordered images. Authors employ feature rearrangement to take advantage of the connections between inner and inter-set pictures. As a result, discussed system can support both verification and open-set identification.

MedFuseNet is a multimodal deep learning network for VQA on medical pictures that is attention-based. The author's of the paper [17] seeks to maximize learning while minimizing complexity by breaking down the issue statement into smaller tasks and guessing the answer. To assess its performance, authors ran a comprehensive set of quantitative and qualitative assessments.

This paper [18] discusses the a typical visual question answering solution is made up of three parts: feature extraction from a single modality, feature fusion between visual and textual channels, and response prediction using the joint representation. We offer an abstract fusion framework that can be used to the vast majority of current VQA models.

A solution to implement visual question answering systems. This model focuses on both image and question attention.The model proposed in this paper [19] not only extracts important features from the image but also tries to understand what part of the question is important via question attention. The hierarchical co-attention mechanism is used to extract the features and create attention maps. Hierarchical question representation is used at the word level , phrase level, and at sentence level of the question. This paper focuses on using the question attention along with image attention and create attention maps . resnet1 is used for image feature extraction , lstm is used for extracting semantic features from the question and the hierarchical attention mechanism is used for identifying the important parts in the question as well as the image . These attention maps obtained are further sent into a recurrent convolutional layer followed by softmax layer inorder to produce the output.The architecture is simple to understand. The use of resnet18 has proven to show better results for this model.

The author's of the paper [20] talks about the major challenges faced while implementing the vqa , such as the lack of enough training data, training of the model is computationally and cost-wise very high, shifting the

models used on the general vqa dataset to the medvqa dataset is quite challenging as the medical images and questions are very different from the general images. The context of medical images and their corresponding questions need different approaches to be understood.

Chapter 3

Exploratory Data Analysis (ImageCLEF 2019)

The ImageCLEF-2019 dataset is used as it is amongst the biggest, open-sourced dataset available for the Med-VQA task. This dataset consists of radiology scans of various body parts (like chest, brain, lungs, spine, etc.). Moreover, it also features radiology scans of different types - X-rays, CT scans, angiography, and many more. The data set consists of a repository of images and their respective question and answer pairs. [14]. It has a train-test split: the training dataset has 3,700 images and 14,800 question-answer pairs associated with it and the test dataset has 501 images and 2,004 question-answer pairs.



Figure 3.1: Image Repository

```
synpic41148|what kind of image is this?|cta - ct angiography
synpic43984|is this a t1 weighted image?|no
synpic38930|what type of imaging modality is used to acquire the image?|us - ultrasound
synpic52143|is this a noncontrast mri?|no
synpic20934|what type of image modality is this?|xr - plain film
synpic19141|with what modality is this image taken?|xr - plain film
synpic25592|what was this image taken with?|xr - plain film
synpic38317|is this a contrast or noncontrast ct?|noncontrast
synpic22829|was the ct scan taken with contrast?|yes
synpic39477|what is the mr weighting in this image?|t2
synpic54866|what type of imaging modality is shown?|ct w/contrast (iv)
synpic60053|what type of imaging does this represent?|xr - plain film
synpic28004|is this a ct scan?|yes
synpic51949|is this an mri image?|no
synpic45041|was gi contrast given to the patient?|no
synpic13026|was gi contrast given to the patient?|no
synpic52613|what was this image taken with?|xr - plain film
synpic21981|is this a ct scan?|no
synpic53380|what imaging modality was used to take this image?|mr - flair
synpic21144|what modality was used to take this image?|mammograph
synpic20556|is this an mri image?|no
synpic39156|is this a contrast or noncontrast mri?|noncontrast
synpic41586|is this a ct scan?|yes
synpic36892|what imaging modality was used to take this image?|ct with iv contrast
synpic20868|is this a ct scan?|yes
synpic27646|what type of image modality is this?|xr - plain film
synpic21181|what type of image modality is seen?|mammograph
synpic19981|was gi contrast given to the patient?|no
synpic54237|what imaging modality was used to take this image?|mr - flair
synpic33704|what type of imaging is this?|us - ultrasound
synpic43008|is this a ct scan?|no
```

Figure 3.2: Question Answer Pairs

In order to better understand the data, some analysis has been done on it in order to gain insights. It has been observed that for each image there is a set of 4 questions associated with it. All 4 questions belong to different categories that are identified as Modality, Plane, Organ, Abnormality categories. Figure 3 shows all the question-answer pairs associated with a specific image.

As can be inferred from the image, “what modality is used to take this image?” belongs to modality class as this category is mainly concerned with the type of the radiology scan (X-ray, CT scans, etc.); “what plane is demonstrated?” falls into the plane category and is concerned with the planarity of the organ shown; “what is the organ system in this image?” refers to organ category and it describes the organ being displayed; the last question refers to the abnormality category as it is concerned with the type of abnormality that is on display in the image.

	questions	answers
1287	what modality is used to take this image?	xr - plain film
4487	what plane is demonstrated?	lateral
7687	what is the organ system in this image?	breast
10887	what is abnormal in the x-ray?	topic: lumbar instability: functional and anat...

Figure 3.3: QA pairs with respect to a single image

Another observation made during the EDA process was that the data is equally distributed among all different categories. This implies that there is no bias towards a certain category.

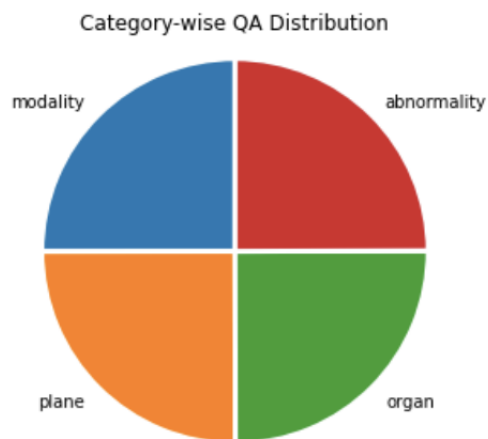


Figure 3.4: Category-wise QA distribution

Chapter 4

Requirement Specification

4.1 Project Perspective

The proposed method is to create a Diagnostic Decision Support System (DDSS) . The input given to this system would be a medical image and a corresponding question associated with that image. This subsequently generates a natural language response as the output. DDSS makes use of a Visual Question Answering (VQA) mechanism which functions at the convergence of computer vision and natural-language-processing methods. A mechanism of this worth would give a helping hand to the medical practitioners and be of service to the common population in solving their complex questions.

4.2 Software Requirement

- OS: Windows/Linux/MacOS
- Language: Python
- Frameworks: Pytorch, Spacy, Keras, OpenVQA

4.3 Hardware Requirement

- A desktop/laptop with active internet connection
- 4GB RAM or more
- 1.2 GHz processor
- i5 processor or more
- Cloud Environment: Google collab

4.4 Functional Requirements

- The proposed algorithm should be able to generate a natural language response given the inputs.
- The inputs required are, a clinical image and a natural language based question related to the image.

4.5 Non-Functional Requirements

4.5.1 Usability Requirement

The system shall allow the users to access the system from a desktop with a web browser and an active internet connection. The system uses a web browser as an interface. Since all users are familiar with the general usage of internet browsing, no special training is required. The system is user friendly which makes the system easy to use.

4.5.2 Availability Requirement

The system is available 100% for the user and is used 24 hrs a day and 365 days a year. The system shall be operational 24 hours a day and 7 days a week.

4.5.3 Efficiency Requirement

Mean Time to Repair (MTTR) - Even if the system fails, the system will be recovered back up within an hour or less.

4.5.4 Accuracy

The system should accurately provide real time information taking into consideration various concurrency issues. The system shall provide 100% access reliability.

4.5.5 Performance Requirement

The information is refreshed depending upon whether some updates have occurred or not in the application. The system shall respond to the member in not less than two seconds from the time of the request submission. The system shall be allowed to take more time when doing large processing jobs. Responses to view information shall take no longer than 5-10 seconds to appear on the screen.

4.5.6 Reliability Requirement

The system must be 100% reliable due to the importance of data and the damages that can be caused by incorrect or incomplete data. The system will run 7 days a week, 24 hours a day.

Chapter 5

Framework and System Design

Visual Question Answering (VQA) in the Medical Domain (MedVQA) is a class of Question-Answering systems that employ image feature extraction mechanisms as well as NLP-based question encoders to answer queries of the end-user with respect to medical imagery (CT, X-Ray and other radiology scans). Such systems usually operate at the junction of vision and natural language techniques to contextually “understand” images so as to answer the image-oriented questions of the user. [20]. An additional module for fusing the features of the two inputs, the image, and the question, may be required as part of the system framework as well.

Feature extraction plays a crucial part in determining the performance of MedVQA systems. Whilst features can be extracted and directly used, as is, in their raw vectorized formats in a MedVQA system, it should be noted, however, that, large vectors require higher computation power for processing. Moreover, larger modalities mean longer runtimes required for training the models. Thus, it is important to reduce the resolution of feature maps before they’re passed into the subsequent modules whilst retaining their internal representations. Pooling strategies are adopted for this purpose internally within the codebase of this project at appropriate places.

Another important method covered as part of the proposed project is that of feature fusion.[21] Just like feature extraction, the fusion of the two extracted features is an essential module when it comes to building accurate MedVQA models. The project proposed in this report uses a mathematical measure called affinity matrix, also known as similarity matrix, for merging the two features. The affinity matrix enables the conversion of the image feature space into a vector space that can be easily connected with the question feature space and vice-versa is also true if we use the transpose of the affinity matrix.

Alongside feature fusion mechanisms, many MedVQA architectures today feature an attention mechanism. This attention mechanism is used to provide extra information to the model - it tells what portions of the input image are important or noteworthy. This extra piece of information provides additional context to the model that results in improved accuracy scores for the model. The proposed project uses a parallel co-attention mechanism that is applied to both the image and question features.

To sum up, in general, a MedVQA system comprises of the following three modules:

- Image feature extraction module
- Question semantic encoder
- Feature fusion module

The sections below present a discussion of the concepts involved for each of the three modules. Sections 5.1 and 5.2 explains the utility of the two feature extraction modules followed by section 5.3 which expands upon the concept of feature fusion.

5.1 Image Feature Extraction Module

Raw images cannot be used directly in a deep learning model which is why the need to extract features arises. Feature extraction is the process in which an image, in this case, a grayscale radiology image, is usually passed through a Convolutional Neural Network (CNN) in order to extract inherent features of the image. Such features are important as they encapsulate the finer details of the image like the boundaries, edges, or contours of the organ displayed in the radiology scan. These extracted features are usually in the form of vectors that can then be mathematically worked upon in the further stages of the architectural pipeline. Conventionally, fully-connected layers are used in conjunction with CNN's to get the final output for classification tasks, however, since we are computing features only to fuse them later on, we do not require fully-connected layers right after the feature extraction module.

5.2 Question Semantic Encoder

This module is responsible for extracting features from sequential data. A sentence in the English language, or in any language, can be considered as sequential data because a certain order is to be maintained in order to interpret the meaning of the sentence - words are read from left to right because they are ordered in that fashion and their "sequence" is important to understand the sentence. A question semantic encoder reads the input natural language text (here, input question) either unidirectionally or bidirectionally. This allows the encoder to interpret the sentence in the specified order and understand the context of each word.

5.3 Feature Fusion

Once the feature maps for both - image and the question text - have been extracted, both features are then merged together into a single entity. This step is required in MedVQA systems (or VQA systems in general) so as to combine the contexts from both inputs.

A naive approach for carrying out multi-modal feature fusion would be to compute the point-wise multiplication between a set of features.^[21] Another relatively simple approach would be to fuse the modalities using simple concatenation. Whilst these methods were adopted in the early years of the MedVQA domain, it has been observed that there are two glaring drawbacks of adopting the above-mentioned approaches.

Firstly, it has been observed that the fused features obtained by employing the above 2 methods have led to outputs that are not able to represent the high-level interactions between the two modalities. The main reason for fusing features in the first place is to be able to capture the relationships or interactions between the two feature maps. This apparent shortcoming of the fusion technique could result in inaccurate outputs. Accuracy scores could take a severe plunge if interactions amongst feature maps aren't represented appropriately by the fused modalities.

Secondly, a lot of computation power is required in order to carry out the above-mentioned two approaches. As discussed earlier, the raw vector outputs from each feature extraction module have very high dimensions. Due to this large size, performing direct operations (like vector concatenation or point-wise multiplication) takes a toll on the machines.

High amounts of computation power, as well as proper utilization of computing resources, are required to perform such resource-intensive operations. A better solution, as proposed in recent papers, is to use bilinear pooling strategies to reduce the dimensions of the extracted vectors.

There are several techniques to carry out feature fusion. The scope of this project report is focused on discussing, analyzing, and interpreting architectures of Attention-based models. Two architectures are discussed in the following sections:

- Multi-modal Factorized Bilinear Pooling model with Co-attention
- Hierarchical Question-Image Co-attention model

The below sections discuss the architecture of the two models and provide a thorough explanation of the algorithms that are involved in the architecture.

5.3.1 Multi-modal Factorized Bilinear Pooling model with Co-attention

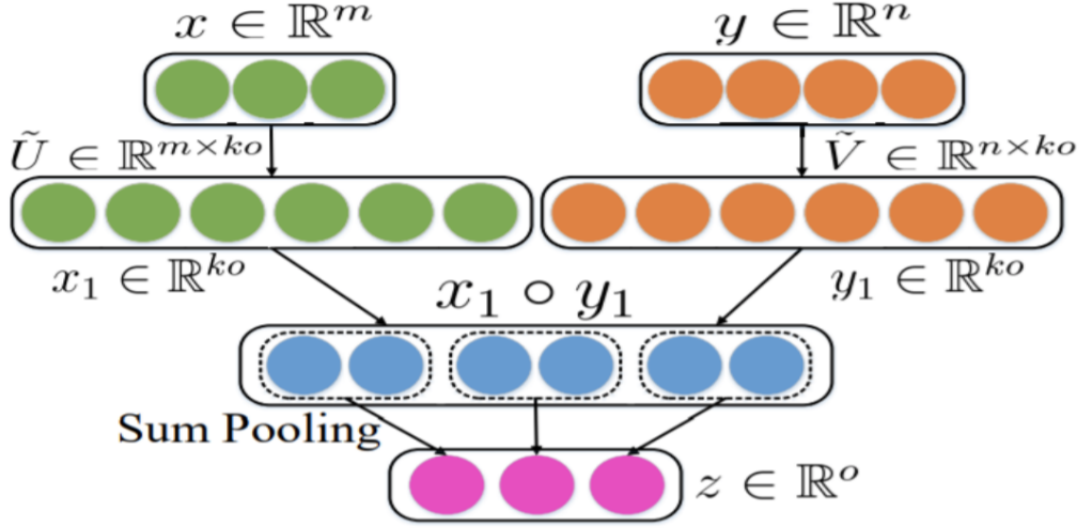


Figure 5.1: MFB architecture

Although bilinear pooling models have been demonstrated to outperform typical linear models (concatenation or point-wise multiplication) for VQA, their high-dimensional representations and computational complexity may severely restrict their practical usefulness. [22] This project report covers a Multi-modal Factorized Bilinear (MFB) pooling strategy for multi-modal feature fusion in order to efficiently and effectively merge multi-modal information.

Figure 5 depicts the comprehensive architecture of MFB. By combining certain regularly used layers, such as fully-connected, element-wise multiplication, and pooling layers, the technique can be easily implemented. Because elementwise multiplication is used, the amplitude of the output neurons might fluctuate substantially, and the model can converge to an undesirable local minimum. Thus, a dropout layer is also placed after the element-wise multiplication layer to prevent over-fitting.

Different questions about the same image might provide completely different answers. As a result, an image attention model that predicts the importance of each spatial grid to the question is useful for predicting the correct response. Attention models, also known as attention mechanisms, are neural network input processing strategies that allow the network to focus on certain parts of a complicated input one at a time until the entire dataset is classified. The idea is to divide complex activities into smaller regions of focus that are processed sequentially. Similar to how the human mind handles a new problem by breaking it down into smaller tasks and tackling them one at a time. In reality, attention enables neural networks to simulate the visual attention process used by humans. The model, like individuals processing a visual picture, analyses a specific spot of an image with intense, "high resolution" concentration while observing the surrounding areas in "low resolution," then modifies the focused point as the network learns about the situation. This attention part of the MFB architecture acts as "additional context" for the model that leads to better accuracy scores.

5.3.2 Hierarchical Question-Image Co-attention model

In the previous works that have been done in the VQA domain, most of the focus is on visual attention. This architecture aims to provide symmetry between the image and the question attention. Image representation is used to guide the question attention and question representation is used to guide the image attention.

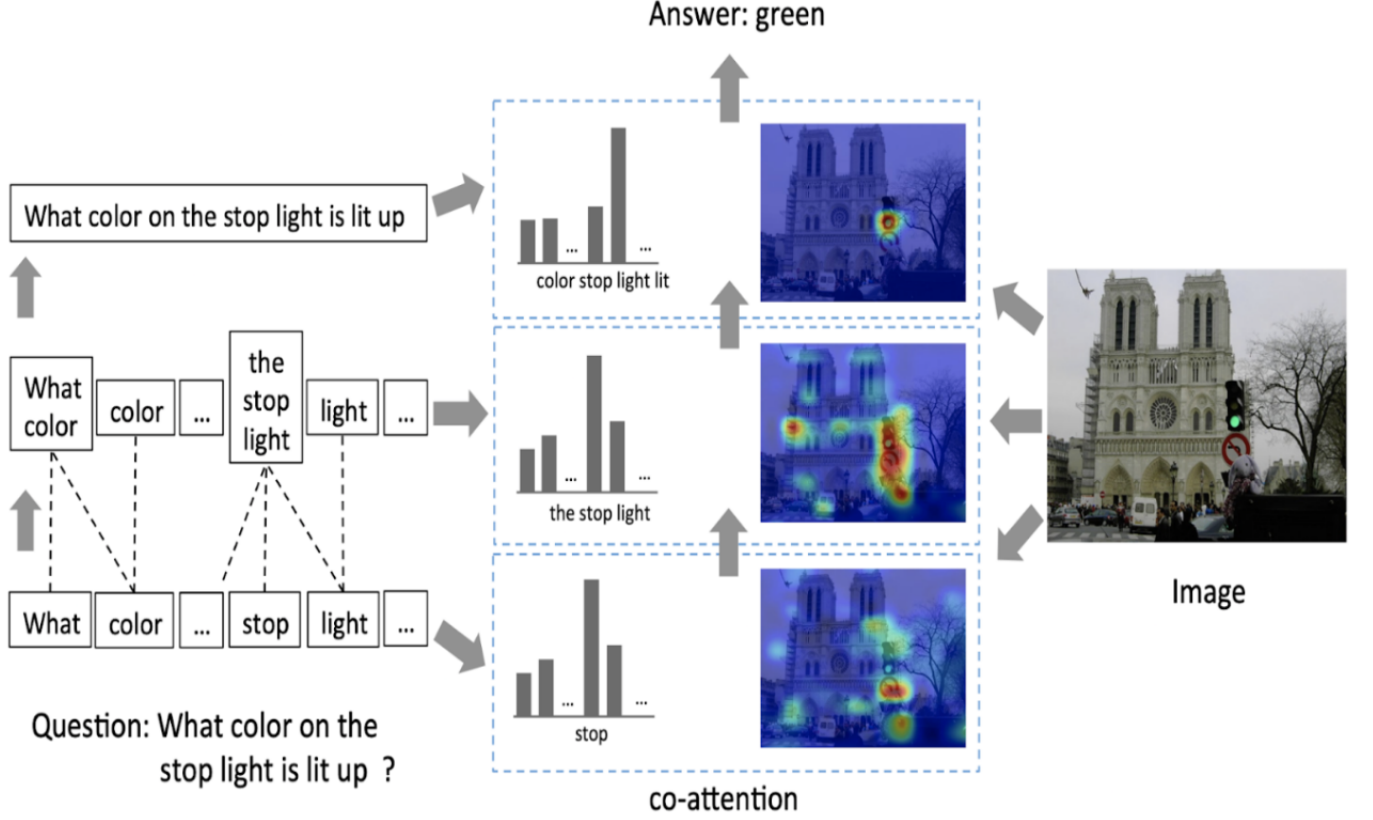


Figure 5.2: Hierarchical Question-Image Co-attention model framework

The present architecture focuses on co-attending the image and the question at 3 levels: word level, phrase level, and sentence level. Question embeddings are extracted at these three levels. At the word level, the embedding is extracted for all the words, and a vector is generated. At the phrase level, the information is extracted from unigrams, bigrams, and trigrams by using convolutional neural networks. The framework of the hierarchical co-attention model is represented in figure 6. It is explained by considering a general question as an example. Word level, phrase level, and sentence level embeddings are extracted for the given question. At each level, co-attention is applied to both the image and question. These co-attended image-question attention maps are then combined together to get the final answer prediction.

As seen in the above architecture, initially the question embedding is extracted at the word level. Corresponding image features are also extracted, and these questions and image embeddings are sent into the co-attention mechanism. This in turn helps in identifying the important part in the image with respect to the question. As we move up in the hierarchy we observe that the attention mechanism focuses on the important part of the question more vividly.

The visual and question components are combined utilizing a concurrent co-attention method. The mathematical concept of affinity matrix is employed here, which serves as a similarity matrix because the visual and question features are relatively distinct from one another. The affinity matrix is produced by calculating the similarity between the image and the question features at each position. The primary considerations for the affinity matrix include a tanh activation function which takes as inputs question features at a certain position, visual features, and the allocated weights. Equation 5.1 shows the mathematical representation of an affinity matrix.

$$C = \tanh(Q^T * W_b * V) \quad (5.1)$$

The parallel attention mechanism is used to calculate the attention of the image and the question at the same time. After that, the attention matrix is used as a feature to predict the image and question attention maps. The affinity matrix aids in the conversion of the question attention space to the image attention space.

It also switches the image attention space to the question attention space.

$$H^v = \tanh(W_v * V + (W_q * Q) * C), H^q = \tanh(W_q * Q + (W_v * V) * C^T) \quad (5.2)$$

$$a^v = \text{softmax}(w_{hv}^T * H^v), a^q = \text{softmax}(w_{hq}^T * H^q) \quad (5.3)$$

In 5.3, a^v and a^q represent the attention probabilities for each image region and question region. The image and question vectors are calculated using the weighted sum of the image and question feature.

$$h^w = \tanh(W_w * (q^w + v^w)) \quad (5.4)$$

$$h^p = \tanh(W_p * [(q^p + v^p), h^w]) \quad (5.5)$$

$$h^s = \tanh(W_s * [(q^s + v^s), h^p]) \quad (5.6)$$

$$p = \text{softmax}(W_h * h^s) \quad (5.7)$$

This VQA task is treated as a classification task because it predicts the model's answers. At each hierarchical level, image and question attention are calculated and combined, along with their respective weights. At each hierarchical level, attention maps are computed. The output of the lower hierarchical level attention maps are sent as inputs to higher hierarchical level attention maps. A multi-layer perceptron is used to encode these attention maps, and the softmax activation function is used to calculate the probability of the final answer.

Chapter 6

Implementation

As discussed earlier, two approaches have been identified to implement MedVQA systems: T

- Multi-modal Factorized Bilinear Pooling model with Co-attention
- Hierarchical Question-Image Co-attention model

In order to implement each of these models, different algorithms have been used for each of these approaches. The VQA model receives a radiology image and a corresponding question associated with it. The output would be a natural language answer.

In the first approach, we use VGG16 for image feature extraction and BERT (Bidirectional Encoder Representation from Transformers) for encoding the question. After the image and the question features are obtained from the previous two modules, they are sent into the feature fusion module. Feature Fusion is done by using Multi-modal Factorized Bilinear Pooling.

The image features are extracted using VGG16. VGG16 is a pre-trained model on the ImageNet Dataset. Originally the VGG16 network consists of 6 convolutional blocks (16 convolutional layers) followed by a fully connected layer. In our implementation, features have been extracted from each individual block of the network. Global Average Pooling has been performed on the extracted features individually. These features are then concatenated to form a 1x1x1984 dimensional tensor.

Question Semantic Encoder is used to extract the features from the question. This is done by using BERT(Bidirectional Encoder Representation From Transformers). Base Bert is used in extracting features of the question. The features are extracted from the last and the penultimate layers of the Bert transformer. The features obtained from these two layers are averaged to obtain a 768-dimensional tensor.

The most challenging part of VQA is understanding both the visual and question features. Since these features vary widely (visual and question features), we use something called as an attention mechanism. Feature fusion helps in merging independent features together so as to process them easily.

Multi-modal Factorized Bilinear Pooling aids the model to get a better understanding of the visual and the question features. Feature Fusion alongside co-attention is a strategy to combine the visual and semantic features. Multi-modal Factorized Bilinear Pooling along with co-attention is used to fuse the modalities of the two features. Weights are assigned to the features based on their importance. This will ensure that the important features are given enough priority and that the model learns them. [15]. Multi-modal Factorized Bilinear Pooling mostly involves concatenation or element-wise summation of the features obtained.

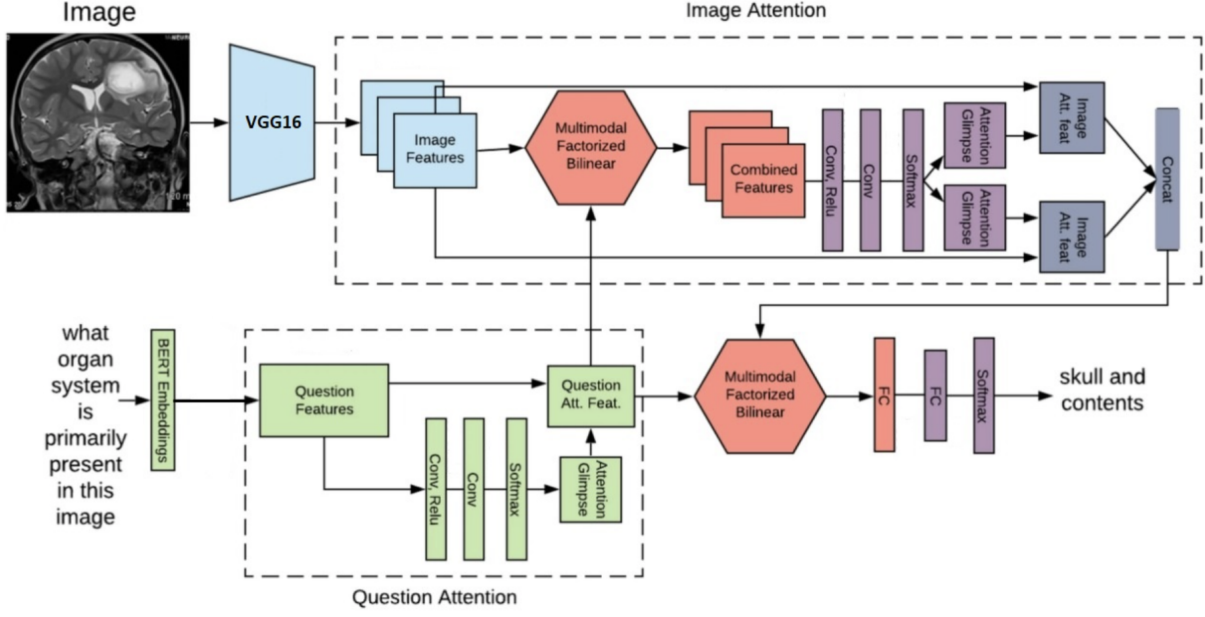


Figure 6.1: MFB with Co-Attention model architecture using VGG16 and BERT

The medical image and the corresponding question are given as input to the model. The features of the image are extracted using VGG16, the question features are extracted using BERT. After the features of both the image and the question are extracted, the question attention mechanism is applied to the question features. The attended question features along with the image features are then passed to the image attention mechanism. From here we receive the attended image features.[17].The attended image and the question features are then combined using Multi-modal Factorized Bilinear Pooling for the answer prediction. In order to Implement the Visual Question Answering system, majorly two main approaches are used. Algorithms like VGG16 and ResNet18 have been used for implementing the Image Feature Extraction Module. Bert, LSTM have been used for Extracting Features from the Question. Multi-modal Factorized Bilinear Pooling, Hierarchical Co attention Mechanisms are used for fusing the features.

6.1 Approach 1: Multi-modal Factorized Bilinear Pooling model with Co-attention

Here, we start by taking a deeper look at the code implementation of approach 1.

Initializing the VGG16 model with pre-trained weights based on the ImageNet dataset. Creating a custom model to extract the features from a specific block. We set it's input layer the same as that of the base vgg16 model. It's output layer can be set as the output of any layers from the vgg16 model. Here we fetch the output from the 2nd convolutional layer in the 1st block.

```

Custom VGG16 Model Analysis

Initializing the VGG16 model with pretrained weights based on ImageNet dataset.

[ ] base_model = VGG16(weights="imagenet")

Creating a custom model to extract the features from a specific block.
We set it's input layer same as that of the base VGG16 model.
It's output layer can be set as the output of ANY layers from VGG16 model. Here, we fetch the output from the 2nd Convolutional Layer in the 1st Block.

[ ] model = Model(inputs=base_model.input, outputs=base_model.get_layer('block1_conv2').output)

[ ] plot_model(model, to_file='custom_vgg.png')

```

Figure 6.2: VGG16 model (with custom output layers)

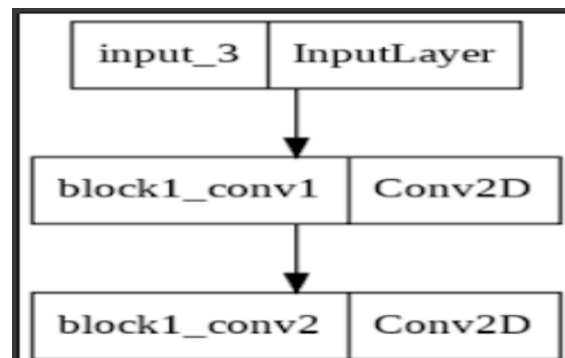


Figure 6.3: Block diagram for VGG16 model with block-1 convolutional-2 output layer

Getting features from each individual blocks by creating a custom vgg16 function.

```

def get_block_features(block_number):
    temp_model = None

    if block_number == 1:
        temp_model = Model(inputs=base_model.input, outputs=base_model.get_layer('block1_conv2').output)
        return temp_model.predict(x)

    elif block_number == 2:
        temp_model = Model(inputs=base_model.input, outputs=base_model.get_layer('block2_conv2').output)
        return temp_model.predict(x)

    elif block_number == 3:
        temp_model = Model(inputs=base_model.input, outputs=base_model.get_layer('block3_conv3').output)
        return temp_model.predict(x)

    elif block_number == 4:
        temp_model = Model(inputs=base_model.input, outputs=base_model.get_layer('block4_conv3').output)
        return temp_model.predict(x)

    elif block_number == 5:
        temp_model = Model(inputs=base_model.input, outputs=base_model.get_layer('block5_conv3').output)
        return temp_model.predict(x)

    elif block_number == 6:
        temp_model = Model(inputs=base_model.input, outputs=base_model.get_layer('block5_pool').output)
        return temp_model.predict(x)

    else:
        print("Error: Invalid block number.")

```

Figure 6.4: Code snippet for custom VGG16 model

Below figure represent the size of individual blocks obtained from the custom VGG16 function. Global Average pooling is applied on the features obtained from these individual blocks and further concatenated.

```
print(block1_features.shape)
print(block2_features.shape)
print(block3_features.shape)
print(block4_features.shape)
print(block5_features.shape)
print(block6_features.shape)

(1, 224, 224, 64)
(1, 112, 112, 128)
(1, 56, 56, 256)
(1, 28, 28, 512)
(1, 14, 14, 512)
(1, 7, 7, 512)
```

Figure 6.5: Output for individual VGG16 block layers

Here, we use the “BERT Base” model alongside its preprocessing model as per the official TensorFlow documentation.

```
[ ] tfhub_handle_encoder = 'https://tfhub.dev/tensorflow/bert_en_uncased_L-12_H-768_A-12/3' # BERT encoder model
tfhub_handle_preprocess = 'https://tfhub.dev/tensorflow/bert_en_uncased_preprocess/3' # BERT preprocessing model

print(f'BERT model selected : {tfhub_handle_encoder}')
print(f'Preprocess model auto-selected: {tfhub_handle_preprocess}')

BERT model selected : https://tfhub.dev/tensorflow/bert_en_uncased_L-12_H-768_A-12/3
Preprocess model auto-selected: https://tfhub.dev/tensorflow/bert_en_uncased_preprocess/3

hub.KerasLayer(): It is used to wrap the saved model into a Keras layer.
In this case, we wrap the BERT preprocessing model into a Keras layer.

[ ] bert_preprocess_model = hub.KerasLayer(tfhub_handle_preprocess)
```

Figure 6.6: Importing base BERT model

The Bert model is wrapped into a Keras layer. The preprocessed text is passed into the bert encoder model and it fetches the results from the last and the penultimate layer.

```
We start by wrapping the BERT Encoder model into a Keras layer.

[ ] bert_model = hub.KerasLayer(tfhub_handle_encoder)

We pass the preprocessed text into the bert encoder model and fetch the results from the last and penultimate layers.

bert_results = bert_model(text_preprocessed)

bert_last_layer = bert_results["encoder_outputs"][-1]
bert_penultimate_layer = bert_results["encoder_outputs"][-2]
print(bert_last_layer.shape)
print(bert_penultimate_layer.shape)
```

Figure 6.7: Processing BERT end result

The final result obtained from the bert encoder is a 768-dimensional tensor.

```

print(result1.shape)
print(result1)

(1, 128, 768)
[[[-0.78652704  0.32944506 -0.63347924 ... -0.5159604  -0.02301432
   0.26579255]
 [ 0.9281898  1.3219947  0.17934746 ... -0.42555523  0.44262075
  -0.17138335]
 [-0.72876966  0.58277035  0.01902795 ...  0.18465838 -0.00838665
  -0.49491024]
 ...
 [ 0.47007158  0.6210689  0.5784095  ... -0.37159032 -0.18816018
   0.30970842]
 [-0.05151443  0.60759974  0.53674793 ... -0.30779076 -0.4174086
   0.37087354]
 [-0.20718032  0.25423297  0.3103047  ... -0.51423943 -0.38613734
   0.2928851  ]]]

```

Figure 6.8: Semantic encodings obtained from BERT

The image and the question features along side other parameters are sent into the Multi-modal Factorized Bilinear Pooling model which results in co attended image and question maps.

```

class MFB_CoAtt(nn.Module):
    def __init__(self, embedding_size, LSTM_units, LSTM_layers, feat_size,
                  batch_size, ans_vocab_size, global_avg_pool_size,
                  dropout = 0.3, mfb_output_dim = 5000):
        super(MFB_CoAtt, self).__init__()
        self.batch_size = batch_size
        self.ans_vocab_size = ans_vocab_size
        self.mfb_output_dim = mfb_output_dim
        self.feat_size = feat_size
        self.mfb_out = 1000
        self.mfb_factor = 5
        self.channel_size = global_avg_pool_size
        self.num_qes_glimpse = 2
        self.num_img_glimpse = 2

        self.LSTM = nn.LSTM(input_size=embedding_size, hidden_size=LSTM_units,
                             num_layers=LSTM_layers, batch_first=False)
        self.pool2d = AvgPool2d(global_avg_pool_size, stride=1)
        self.Dropout = nn.Dropout(p=dropout, )
        self.Linear_img_proj = nn.Linear(self.feat_size, self.mfb_output_dim)
        self.Linear_qes_proj = nn.Linear(LSTM_units, self.mfb_output_dim)
        self.Linear_predict = nn.Linear(self.mfb_out, ans_vocab_size)
        self.Softmax = nn.Softmax()

        self.Linear1_q_proj = nn.Linear(LSTM_units*self.num_qes_glimpse, self.mfb_output_dim)
        self.Linear2_q_proj = nn.Linear(LSTM_units*self.num_qes_glimpse, self.mfb_output_dim)
        self.Linear_i_proj = nn.Linear(self.feat_size*self.num_img_glimpse, self.mfb_output_dim)
        self.Conv_i_proj = nn.Conv2d(self.feat_size, self.mfb_output_dim, 1)

```

Figure 6.9: Code implementation for MFB

6.2 Approach 2: Hierarchical Question-Image Co-attention model

In the second approach, Resnet18 is used for image feature extraction, lstm is used for extracting the semantic features from the question. Resnet18 is a pre-trained model on the Imagenet dataset. LSTM is used to extract the semantic question features. A hierarchical question representation is used while extracting the embeddings from the questions. Question embeddings are extracted at 3 levels: the word level, phrase level, and sentence level. At each of these levels, the image and the question attention are calculated and combined together which would result in the final answer.[\[23\]](#) A parallel co-attention mechanism is used for fusing the visual and the question features. Since the visual and the question features are quite different from each other we use an affinity matrix, which acts as a similarity matrix.

The similarity between the image and the question features is calculated at all locations, this results in the affinity matrix. The affinity matrix is a tanh activation function that primarily takes into consideration question features at a particular location, visual features along with the assigned weights. Below code snippet is representative of equation 5.1 discussed in chapter 5.

```
C = torch.matmul(Q, torch.matmul(self.W_b, V)) # B x L x 196
```

Figure 6.10: Code implementation for calculating affinity matrix

The attention of the image and the question are calculated simultaneously using the parallel attention mechanism.[19] The attention matrix is then used as a feature which will in turn be used to predict the image and the question attention maps. The affinity matrix helps in transforming the question attention space into the image attention space. It also transforms the image attention space into the question attention space Again, the below code represents equations 5.2 and 5.3 as previously discussed.

```
def parallel_co_attention(self, V, Q): # V : B x 512 x 196, Q : B x L x 512

    C = torch.matmul(Q, torch.matmul(self.W_b, V)) # B x L x 196

    H_v = self.tanh(torch.matmul(self.W_v, V) + torch.matmul(torch.matmul(self.W_q, Q.permute(0, 2, 1)), C)) #
    H_q = self.tanh(torch.matmul(self.W_q, Q.permute(0, 2, 1)) + torch.matmul(torch.matmul(self.W_v, V), C.permute(0, 2, 1))) #

    a_v = F.softmax(torch.matmul(torch.t(self.w_hv), H_v), dim=2) # B x 1 x 196
    a_q = F.softmax(torch.matmul(torch.t(self.w_hq), H_q), dim=2) # B x 1 x L

    v = torch.squeeze(torch.matmul(a_v, V.permute(0, 2, 1))) # B x 512
    q = torch.squeeze(torch.matmul(a_q, Q)) # B x 512

    return v, q
```

Figure 6.11: Code implementation for computing attention feature maps

Attention probabilities for each image region and question region are represented by a^v and a^q . The weighted sum of the image and the question feature is used to calculate the image and the question vectors respectively. In order to predict the answers of the model, the VQA is considered a classification task. Image and Question attention is calculated at each hierarchical level and combined along with their respective weights. Attention maps are calculated at each of the hierarchical levels. The attention maps at the lower hierarchical level are sent as inputs to the attention maps of the higher hierarchical level.

A multi-layer perceptron is used to encode these attention maps, and also get the probability for the final answer using the softmax activation function similar to what was shown in equations 5.4-5.6.

```

h_w = self.tanh(self.W_w(q_word + v_word))
h_p = self.tanh(self.W_p(torch.cat(((q_phrase + v_phrase), h_w), dim=1)))
h_s = self.tanh(self.W_s(torch.cat(((q_sent + v_sent), h_p), dim=1)))

logits = self.fc(h_s)

```

Figure 6.12: Code implementation for computing the final output using the hierarchical representation

An answer dictionary is created, which consists of answer ids for respective questions and the answers as well.[20] Given an image and a question corresponding to that image, the answer is selected from a possible set of answers. This is done with the help of argmax function .

When an image and a question are given as input to the model the output generated is an answer id. This answer id is further mapped with the corresponding answer using the answer dictionary.

```

Iteration: 50
Prediction: [1386 259 374 1386 1386 1508 978 374 978 1386 1508 1386 1508 1386
978 1386 1508 374 374 259 1508 1508 978 259 418 1386 361 1508
1508 1464 1386 1386]
Actual answer [ 624 1167 150 328 1090 442 225 1197 1163 624 442 624 442 132
1497 1105 442 1197 150 1035 1044 1385 726 95 470 576 726 442
1385 1513 624 624]

```

Figure 6.13: Output vectors produced by model

The model in the second approach was used for the implementation, due to certain limitations that were encountered in the first approach. The results obtained in the first approach were ambiguous - the MFB architecture was able to successfully merge the two feature maps with applied co-attention but the necessary steps required to obtain a classification answer were not made clear. We speculate that an answer dictionary/vocabulary would be required in order to utilize the output from the MFB module. Moreover, the time taken to train the model in this first approach was much longer than that of the second approach - mostly due to the sheer complexity of the MFB architecture. Due to these technical challenges posed by the first approach, the second approach was adopted as the primary method for implementation of the VQA system. Due to these technical challenges posed by the first approach, the second approach was adopted as the primary method for implementation of the VQA system. In the second approach (hierarchical image-question co-attention model), the architecture's focus was shifted from general VQA to the MedVQA domain (i.e. ImageCLEF 2019 radiology dataset). Based on the learnings from the previous approach, an answer dictionary was implemented for the second approach training the model over the MedVQA dataset. A scheduler function was also used to implement the learning rate decay technique. According to a comparative study, resnet18 has shown much better results than vgg16 for image feature extraction in the VQA domain. Resnet18 is used in the second approach, it has been observed that the use of resnet18 has resulted in much higher accuracy for the model, rather than vgg16. Resnet18 also solves the vanishing gradient problem that exists in other convolutional Neural Networks. The hierarchical question representation has been used for question embedding, which provides a much deeper understanding to the model as the features are extracted at each level of the hierarchy, and co-attention is applied at every level as well. The use of question attention is yet another advantage, as it would help the model better understand the question that is being asked with respect to the image. There has been quite limited focus on the question attention in the models present for VQA. The model in the second approach has a simple architecture and it can be trained relatively more efficiently.

Chapter 7

Testing

The primary focus for testing has been on approach 2.

The ImageCLEF dataset has been split into Train-Test components. For both testing and training, the dataset is loaded into the model using Data Loaders. Dataloaders help in loading and iterating over the dataset. The total size of the training dataset is around 3,700 images and 14,800 QA-pairs and the size of the testing dataset is 501 images and 2,004 QA-pairs.

During training, the model is trained on the training dataset and tested on the test dataset at each epoch. For each epoch, an accuracy score and cross-entropy loss are computed.

The accuracy function enumerates over the valid data set with each iteration consisting of 32 image-question pairs and predicts the answers. It returns the total number of successful classification. The final accuracy is calculated using the formula:

$$accuracy = \frac{\text{Total number of successful classification}}{\text{Length of data loader} * \text{Batch size}}$$

The below loss-epoch graph shows the loss at each epoch without using any particular training strategy.

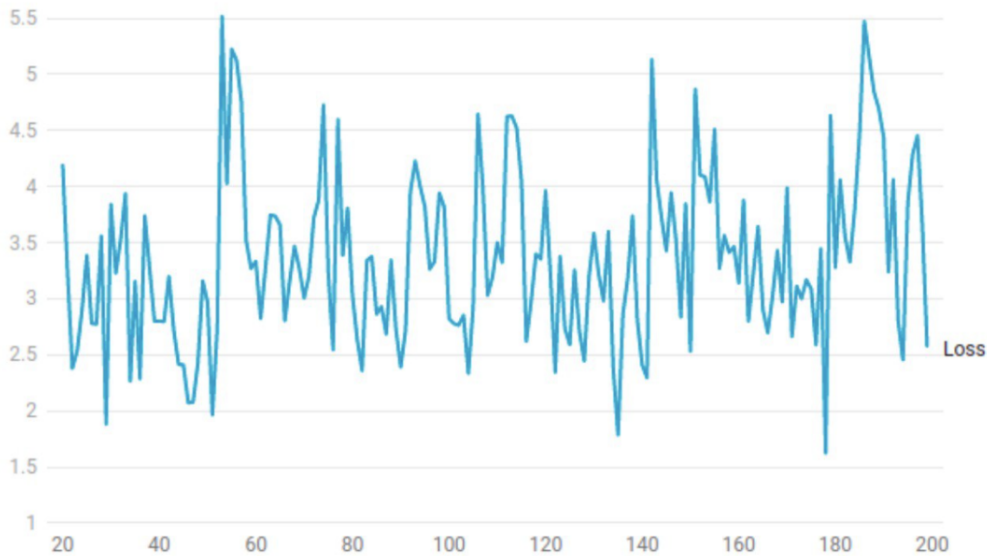


Figure 7.1: Loss-epoch graph obtained during model training

The loss values, at each epoch, for this model are logged in a separate text file which can then be used for analysis later on. The contents of this text file are also shown below for reference.

```

stats.txt
{"epoch": 175, "loss": 3.088129997253418, "time": 540}
{"epoch": 176, "loss": 2.5887038707733154, "time": 541}
{"epoch": 177, "loss": 3.444145917892456, "time": 541}
{"epoch": 178, "loss": 1.625876545906067, "time": 541}
/content/drive/MyDrive/MedVQA/hierarchical/main.py
{"epoch": 179, "loss": 4.631123065948486, "time": 2283}
{"epoch": 180, "loss": 3.2780849933624268, "time": 540}
{"epoch": 181, "loss": 4.05740213394165, "time": 540}
{"epoch": 182, "loss": 3.554409980773926, "time": 541}
{"epoch": 183, "loss": 3.328317403793335, "time": 540}
{"epoch": 184, "loss": 3.791888475418091, "time": 540}
{"epoch": 185, "loss": 4.455935478210449, "time": 540}
/content/drive/MyDrive/MedVQA/hierarchical/main.py
{"epoch": 186, "loss": 5.468970775604248, "time": 1622}
/content/drive/MyDrive/MedVQA/hierarchical/main.py
{"epoch": 187, "loss": 5.153775691986084, "time": 1150}
{"epoch": 188, "loss": 4.843677997589111, "time": 316}
{"epoch": 189, "loss": 4.692816257476807, "time": 318}
{"epoch": 190, "loss": 4.439323902130127, "time": 317}
{"epoch": 191, "loss": 3.2392642498016357, "time": 317}
{"epoch": 192, "loss": 4.06124210357666, "time": 318}
{"epoch": 193, "loss": 2.8026487827301025, "time": 317}
{"epoch": 194, "loss": 2.4563934803009033, "time": 317}
{"epoch": 195, "loss": 3.8541557788848877, "time": 317}
{"epoch": 196, "loss": 4.282835006713867, "time": 317}
{"epoch": 197, "loss": 4.452075481414795, "time": 317}
{"epoch": 198, "loss": 3.661985397338867, "time": 318}
{"epoch": 199, "loss": 2.5803303718566895, "time": 317}
/content/drive/MyDrive/MedVQA/hierarchical/main.py

```

Figure 7.2: Log file for all epochs trained during model training runtime

7.1 Learning Rate Decay Technique

An updated model was then put to training with several tweaks. A scheduler was added to implement the learning rate decay strategy. The learning rate decay technique helps in the optimization and generalization of the model. The scheduler is used to optimize the learning rate. Initially, a larger learning rate is assigned to the model, as the number of epochs increases the scheduler aims at reducing the learning rate and aims to reach local minima. The scheduler is used to Set the patience to 2 (default: 3). Patience is one of the parameters in the scheduler function, which is used to decrease the learning rate if the loss and accuracy of the model do not improve after a certain number of epochs. The intermediate results of this model show a steady descent in the loss curve with a much-improved accuracy score. The below loss-epoch graph shows the loss obtained at each epoch with learning rate decay strategy.



Figure 7.3: Loss-epoch graph obtained during model training using LR-decay strategy

It is observed that the loss has a smooth curve after the changes in the parameters and the implementation of the scheduler.

Using the accuracy formula mentioned above, the final accuracy score for the model after only being trained for 40 epochs was found to be 42%. The model is still under training and will be trained for a total of 200 epochs.

Chapter 8

Results

8.1 Approach 1: Multi-modal Factorized Bilinear Pooling with Co-attention

The first approach was implemented majorly by three modules which are: image feature extraction using VGG16, question semantic encoder using BERT and feature fusion using Multi-modal Factorized Bilinear Pooling. The results obtained from each of these modules has been mentioned below.

8.1.1 Outcome for module 1

Successfully built a VGG-16 network to extract the inherent features from image data and obtained a 1x1x1984 dimensional vector.

```
[ ] result = np.concatenate((block1_gap, block2_gap, block3_gap, block4_gap, block5_gap, block6_gap), axis=1)
    result = np.array([result])
    print(result.shape)
    print(result)

(1, 1, 1984)
[[[141.51372  123.1967  121.80125  ...  8.5662775  0.3047954
    12.977199 ]]]
```

Figure 8.1: VGG16 module outcome

8.1.2 Outcome for module 2

Successfully extracted semantic features from the question using "base BERT" and obtained a 768-dimensional tensor.

```
▶ print(result1.shape)
print(result1)

↳ (1, 128, 768)
[[[-0.78652704  0.32944506 -0.63347924 ... -0.5159604 -0.02301432
   0.26579255]
 [ 0.9281898  1.3219947  0.17934746 ... -0.42555523  0.44262075
  -0.17138335]
 [-0.72876966  0.58277035  0.01902795 ...  0.18465838 -0.00838665
  -0.49491024]
 ...
 [ 0.47007158  0.6210689  0.5784095  ... -0.37159032 -0.18816018
   0.30970842]
 [-0.05151443  0.60759974  0.53674793 ... -0.30779076 -0.4174086
   0.37087354]
 [-0.20718032  0.25423297  0.3103047  ... -0.51423943 -0.38613734
   0.2928851  ]]]
```

Figure 8.2: BERT module outcome

8.1.3 Outcome for module 3

Successfully combined the outputs from the previous two modules using Multi-Modal Bilinear Fusion method.

```
▶ print(mfb_output.shape)
print(mfb_output)

↳ torch.Size([5, 1000])
tensor([[0.0010, 0.0010, 0.0011, ..., 0.0010, 0.0010, 0.0010],
        [0.0009, 0.0010, 0.0010, ..., 0.0010, 0.0010, 0.0010],
        [0.0010, 0.0010, 0.0010, ..., 0.0010, 0.0009, 0.0010],
        [0.0010, 0.0011, 0.0011, ..., 0.0010, 0.0010, 0.0010],
        [0.0010, 0.0009, 0.0010, ..., 0.0010, 0.0010, 0.0010]],
        grad_fn=<SoftmaxBackward0>)
```

Figure 8.3: MFB module outcome

Using approach 1, both the extracted feature maps (image and question features) were successfully fused together but there seems to be no clearly established strategy to proceed further. Our group speculates that there is a need for an answer dictionary to be developed in order to fetch the final classification output from this intermediate result. Moreover, the training for this approach is significantly longer than approach 2. Work is still being carried out for this approach. Future work that can be carried out in this approach would be to build a answer vocabulary and map the obtained tensor with the vocabulary.

8.2 Approach 2: Hierarchical Question-Image Co-attention mechanism

The second approach was implemented using a hierarchical co-attention mechanism. It also used hierarchical question embedding at multiple levels being the word level, phrase level and sentence level. Resnet18 was used for Image Feature Extraction, and LSTM was used for question encoding. Feature fusion was done by using a hierarchical co-attention mechanism. This approach takes into account question attention as well. The hierarchical co-attention mechanism extracts the co-attended image and question features. The results obtained from this approach are listed below.

```
Prediction: [1386 259 374 1386 1386 1508 978 374 978 1386 1508 1386 1508 1386
978 1386 1508 374 374 259 1508 1508 978 259 418 1386 361 1508
1508 1464 1386 1386]
Actual answer [ 624 1167 150 328 1090 442 225 1197 1163 624 442 624 442 132
1497 1105 442 1197 150 1035 1044 1385 726 95 470 576 726 442
1385 1513 624 624]
```

Figure 8.4: Model outcome for approach 2

The output consists of a list of answer ids corresponding to their questions. These answer ids are further mapped from the answer dictionary in order to obtain the answer for the question that was asked.

```
model.py X loss_of_epoch1.txt X
/content/model.py
1 Loss on epoch 0 : 4.827191402316093
2 Accuracy on epoch 0 : 363
3 Loss on epoch 1 : 4.327024203538895
4 Accuracy on epoch 1 : 428
5 Loss on epoch 2 : 4.091721806526184
6 Accuracy on epoch 2 : 473
7 Loss on epoch 3 : 3.9289184272289277
8 Accuracy on epoch 3 : 483
9 Loss on epoch 4 : 3.7313357794284823
10 Accuracy on epoch 4 : 548
11 Loss on epoch 5 : 3.483756142258644
12 Accuracy on epoch 5 : 591
13 Loss on epoch 6 : 3.417079070806503
14 Accuracy on epoch 6 : 550
15 Loss on epoch 7 : 3.4783317309618
16 Accuracy on epoch 7 : 566
17 Loss on epoch 8 : 3.3442553249001503
18 Accuracy on epoch 8 : 641
19 Loss on epoch 9 : 3.259169156849384
20 Accuracy on epoch 9 : 606
21 Loss on epoch 10 : 3.2722210550308226
22 Accuracy on epoch 10 : 548
23 Loss on epoch 11 : 3.136818144917488
24 Accuracy on epoch 11 : 652
25 Loss on epoch 14 : 2.986817311346531
26 Accuracy on epoch 14 : 645
27 Loss on epoch 15 : 2.924548974633217
28 Accuracy on epoch 15 : 639
29 Loss on epoch 16 : 2.8756509897112847
30 Accuracy on epoch 16 : 665
31 Loss on epoch 17 : 2.8414809301495554
32 Accuracy on epoch 17 : 672
33 Loss on epoch 18 : 2.8038976994156837
34 Accuracy on epoch 18 : 700
35 Loss on epoch 19 : 2.8994100764393806
36 Accuracy on epoch 19 : 668
37 Loss on epoch 20 : 2.8196069476008416
38 Accuracy on epoch 20 : 672
39 Loss on epoch 21 : 2.7815997782349586
40 Accuracy on epoch 21 : 688
41 Loss on epoch 22 : 2.747326147854328
```

Figure 8.5: Loss and accuracy scores for approach 2

Based on early testing on the validation dataset, the model had an accuracy of 42%, whereas the state-of-the-art model had an accuracy of 62%. This outcome is based on a model that was trained for only 40 epochs. The model is presently being trained and will be trained for a total of 200 epochs. There is a lot of room for development because there is reason to believe that with adequate training and the right choice of hyperparameters, the model will perform better.

Chapter 9

Conclusion and Future Scope

In this report, two approaches to implement Visual Question Answering (VQA) systems have been discussed. A comparative study of the results obtained from both of these approaches has been carried out. The results obtained in the first approach were ambiguous - the MFB architecture was able to successfully merge the two feature maps with applied co-attention but the necessary steps required to obtain a classification answer were not made clear. We speculate that an answer dictionary/vocabulary would be required in order to utilize the output from the MFB module. Moreover, the time taken to train the model in this first approach was much longer than that of the second approach - mostly due to the sheer complexity of the MFB architecture. Due to these technical challenges posed by the first approach, the second approach was adopted as the primary method for implementation of the VQA system. In the second approach (hierarchical image-question co-attention model), the architecture's focus was shifted from general VQA to the MedVQA domain (i.e. ImageCLEF 2019 radiology dataset). Based on the learnings from the previous approach, an answer dictionary was implemented for the second approach training the model over the MedVQA dataset. A scheduler function was also used to implement the learning rate decay technique. Based on the preliminary testing conducted over the validation dataset, the model showed an accuracy of 42% whilst the state-of-the-art model has an accuracy of 62%. This result is based on the model that was trained only for 40 epochs. The model is currently under training and will be trained for 200 epochs. There is a huge scope for improvement as our assumption is that with proper training and with correct set of hyperparameters, the model could show improved accuracy scores.

Bibliography

- [1] Randolph A Miller. Diagnostic decision support systems. In *Clinical decision support systems*, pages 181–208. Springer, 2016.
- [2] Talya Porat, Brendan Delaney, and Olga Kostopoulou. The impact of a diagnostic decision support system on the consultation: perceptions of gps and patients. *BMC medical informatics and decision making*, 17(1):1–9, 2017.
- [3] N. W. Moukheibir. Universal computer assisted diagnosis. US Patent 6,021,404., 2000.
- [4] Leigang Qu, Meng Liu, Jianlong Wu, Zan Gao, and Liqiang Nie. Dynamic modality interaction modeling for image-text retrieval. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1104–1113, 2021.
- [5] Matteo Stefanini, Marcella Cornia, Lorenzo Baraldi, Silvia Cascianelli, Giuseppe Fiameni, and Rita Cucchiara. From show to tell: A survey on deep learning-based image captioning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pages 1–1, 2022.
- [6] Xuehai He, Zhuo Cai, Wenlan Wei, Yichen Zhang, Luntian Mou, Eric Xing, and Pengtao Xie. Towards visual question answering on pathology images. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pages 708–718, Online, August 2021. Association for Computational Linguistics.
- [7] Qi Wu, Damien Teney, Peng Wang, Chunhua Shen, Anthony Dick, and Anton Van Den Hengel. Visual question answering: A survey of methods and datasets. *Computer Vision and Image Understanding*, 163:21–40, 2017.
- [8] Sheng Zhang, Min Chen, Jincal Chen, Fuhao Zou, Yuan-Fang Li, and Ping Lu. Multimodal feature-wise co-attention method for visual question answering. *Information Fusion*, 73:1–10, 2021.
- [9] Guotai Wang, Wenqi Li, Maria A Zuluaga, Rosalind Pratt, Premal A Patel, Michael Aertsen, Tom Doel, Anna L David, Jan Depreest, Sébastien Ourselin, et al. Interactive medical image segmentation using deep learning with image-specific fine tuning. volume 37, pages 1562–1573. IEEE, 2018.
- [10] Mohammad Havaei, Axel Davy, David Warde-Farley, Antoine Biard, Aaron Courville, Yoshua Bengio, Chris Pal, Pierre-Marc Jodoin, and Hugo Larochelle. Brain tumor segmentation with deep neural networks. volume 35, pages 18–31. Elsevier, 2017.
- [11] Damien Teney, Qi Wu, and Anton van den Hengel. Visual question answering: A tutorial. In *Nature Publishing Group*, pages 63–75. IEEE, 2017.
- [12] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. pages 1735–1780. MIT Press, 1997.
- [13] Deepak Gupta, Swati Suman, and Asif Ekbal. Hierarchical deep multi-modal network for medical visual question answering. In *Expert Systems with Applications*, page 113993. Elsevier, 2021.
- [14] Binh D Nguyen, Thanh-Toan Do, Binh X Nguyen, Tuong Do, Erman Tjiputra, and Quang D Tran. Overcoming data limitation in medical visual question answering. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 522–530. Springer, 2019.
- [15] Zhou Yu, Jun Yu, Jianping Fan, and Dacheng Tao. Multi-modal factorized bilinear pooling with co-attention learning for visual question answering. In *Proceedings of the IEEE international conference on computer vision*, pages 1821–1830. IEEE, 2017.

- [16] Xiaofeng Liu, Zhenhua Guo, Site Li, Lingsheng Kong, Ping Jia, Jane You, and BVK Kumar. Permutation-invariant feature restructuring for correlation-aware image set-based recognition. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4986–4996, 2019.
- [17] Purushotham S. Reddy C.K. Sharma, D. Medfusenet: An attention-based multimodal deep learning model for visual question answering in the medical domain. In *Nature Publishing Group*, pages 1–18. Nature Publishing group, 2021.
- [18] Dongxiang Zhang, Rui Cao, and Sai Wu. Information fusion in visual question answering: A survey. In *Information Fusion*, pages 268–280. Elsevier, 2019.
- [19] Jiasen Lu, Jianwei Yang, Dhruv Batra, and Devi Parikh. Hierarchical question-image co-attention for visual question answering. In *Advances in neural information processing systems*, pages 25–67. NeuroIPS, 2016.
- [20] Haoyue Bai, Xiaoyan Shan, Yefan Huang, and Xiaoli Wang. Mvqas: A medical visual question answering system. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, pages 4675–4679. ACM, 2021.
- [21] Jin-Hwa Kim, Jaehyun Jun, and Byoung-Tak Zhang. Bilinear attention networks. In *Advances in neural information processing systems*, pages 63–75. NIPS, 2018.
- [22] Jin-Hwa Kim, Kyoung-Woon On, Woosang Lim, Jeonghee Kim, Jung-Woo Ha, and Byoung-Tak Zhang. Hadamard product for low-rank bilinear pooling. In *arXiv preprint arXiv:1610.04325*, pages 25–67. arxiv, 2016.
- [23] Dhruv Batra† Devi Parikh† Jiasen Lu, Jianwei Yang. Hierarchical question-image co-attention for visual question answering. In *30th Conference on Neural Information Processing Systems (NIPS 2016), Barcelona, Spain.*, pages 1–7. NIPS, 2016.